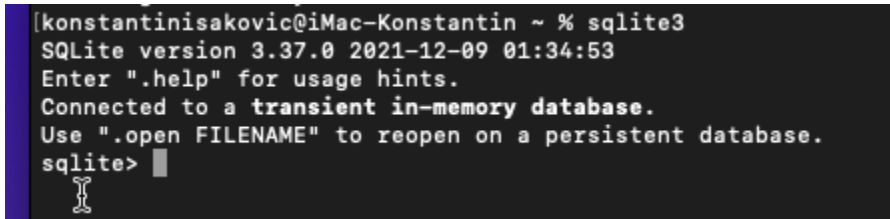


Отчёт по лабораторной работе №3

Задание 1. Установка SQLite

SQLite проверена с помощью команды:

sqlite3



```
[konstantinisakovic@iMac-Konstantin ~ % sqlite3
SQLite version 3.37.0 2021-12-09 01:34:53
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite>
```

Задание 2. Управление базой данных из консоли

Упражнение 2.3. Создание базы данных и выполнение запросов

Описание предметной области

«Абитуриент»: id; фамилия; имя; отчество; пол; национальность; дата рождения (год, месяц число); домашний адрес (почтовый индекс, страна, область, район, город, улица, дом, квартира); оценки по ЦТ; проходной балл.

Создание таблицы

```
CREATE TABLE abiturient (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  last_name TEXT NOT NULL,
  first_name TEXT NOT NULL,
  middle_name TEXT,
  gender TEXT CHECK(gender IN ('M', 'Ж')),
  nationality TEXT,
  birth_date TEXT,
  postal_code TEXT,
  country TEXT,
  region TEXT,
  district TEXT,
  city TEXT,
  street TEXT,
  house TEXT,
```

```

apartment TEXT,
ct_scores TEXT,
passing_score INTEGER
);

```

```

sqlite> CREATE TABLE abiturient (
...>     id INTEGER PRIMARY KEY AUTOINCREMENT,
...>     last_name TEXT NOT NULL,
...>     first_name TEXT NOT NULL,
...>     middle_name TEXT,
...>     gender TEXT CHECK(gender IN ('М', 'Ж')),
...>     nationality TEXT,
...>     birth_date TEXT, -- формат YYYY-MM-DD
...>     postal_code TEXT,
...>     country TEXT,
...>     region TEXT,
...>     district TEXT,
...>     city TEXT,
...>     street TEXT,
...>     house TEXT,
...>     apartment TEXT,
...>     ct_scores TEXT, -- например, "95,87,92"
...>     passing_score INTEGER
...> );

```

Вставка данных

INSERT INTO abiturients VALUES (...);

```

...> );
sqlite> INSERT INTO abiturient (last_name, first_name, middle_name, gender, nationality, birth_date, postal
_code, country, region, district, city, street, house, apartment, ct_scores, passing_score) VALUES
...> ('Иванов', 'Иван', 'Иванович', 'М', 'русский', '2005-03-12', '123456', 'Россия', 'Московская', 'Рам
енский', 'Раменское', 'Ленина', '10', '45', '92,88,91', 271),
...> ('Петрова', 'Анна', 'Сергеевна', 'Ж', 'русская', '2006-07-19', '654321', 'Россия', 'Московская', 'Л
юберецкий', 'Люберцы', 'Кирова', '22', '5', '85,79,83', 247),
...> ('Сидоров', 'Петр', 'Алексеевич', 'М', 'русский', '2005-11-02', '111222', 'Россия', 'Тверская', 'Ка
лининский', 'Тверь', 'Советская', '5', '12', '78,82,80', 240),
...> ('Козлов', 'Алексей', 'Николаевич', 'М', 'русский', '2005-01-25', '333444', 'Россия', 'СПб', 'Примо
рский', 'СПб', 'Парашютная', '8', '2', '96,94,97', 287),
...> ('Васильева', 'Мария', 'Павловна', 'Ж', 'русская', '2006-09-30', '555666', 'Россия', 'Лен. обл.', '
Всеволожский', 'Мурино', 'Шоссе', '1', '34', '88,90,85', 263),
...> ('Михайлов', 'Дмитрий', 'Игоревич', 'М', 'белорус', '2005-05-17', '777888', 'Беларусь', 'Минская',
'Минский', 'Минск', 'Независимости', '15', '99', '91,89,94', 274),
...> ('Новикова', 'Елена', 'Владимировна', 'Ж', 'русская', '2005-12-11', '999000', 'Россия', 'Калужская',
'Обнинск', 'Обнинск', 'Курчатова', '7', '18', '72,68,75', 215);

```

Выборка данных

- Все данные:

SELECT * FROM abiturient;

```
sqlite> SELECT * FROM abiturient;
1|Иванов|Иван|Иванович|М|русский|2005-03-12|123456|Россия|Московская|Раменский|Раменское|Ленина|10|45|92,88,91|271
2|Петрова|Анна|Сергеевна|Ж|русская|2006-07-19|654321|Россия|Московская|Люберецкий|Люберцы|Кирова|22|5|85,79,83|247
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Парашютная|8|2|96,94,97|287
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
7|Новикова|Елена|Владимировна|Ж|русская|2005-12-11|999000|Россия|Калужская|Обнинск|Обнинск|Курчатова|7|18|72,68,75|215
sqlite> SELECT * FROM abiturient;
1|Иванов|Иван|Иванович|М|русский|2005-03-12|123456|Россия|Московская|Раменский|Раменское|Ленина|10|45|92,88,91|271
2|Петрова|Анна|Сергеевна|Ж|русская|2006-07-19|654321|Россия|Московская|Люберецкий|Люберцы|Кирова|22|5|85,79,83|247
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Парашютная|8|2|96,94,97|287
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
7|Новикова|Елена|Владимировна|Ж|русская|2005-12-11|999000|Россия|Калужская|Обнинск|Обнинск|Курчатова|7|18|72,68,75|215
```

- Сортировка по id:

SELECT * FROM abiturient ORDER BY id;

```
sqlite> SELECT * FROM abiturient ORDER BY id;
1|Иванов|Иван|Иванович|М|русский|2005-03-12|123456|Россия|Московская|Раменский|Раменское|Ленина|10|45|92,88,91|271
2|Петрова|Анна|Сергеевна|Ж|русская|2006-07-19|654321|Россия|Московская|Люберецкий|Люберцы|Кирова|22|5|85,79,83|247
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Парашютная|8|2|96,94,97|287
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
7|Новикова|Елена|Владимировна|Ж|русская|2005-12-11|999000|Россия|Калужская|Обнинск|Обнинск|Курчатова|7|18|72,68,75|215
sqlite>
```

- Сортировка по имени:

SELECT * FROM abiturient ORDER BY first_name;

```
sqlite> SELECT * FROM abiturient ORDER BY first_name;
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Парашютная|8|2|96,94,97|287
2|Петрова|Анна|Сергеевна|Ж|русская|2006-07-19|654321|Россия|Московская|Люберецкий|Люберцы|Кирова|22|5|85,79,83|247
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
7|Новикова|Елена|Владимировна|Ж|русская|2005-12-11|999000|Россия|Калужская|Обнинск|Обнинск|Курчатова|7|18|72,68,75|215
1|Иванов|Иван|Иванович|М|русский|2005-03-12|123456|Россия|Московская|Раменский|Раменское|Ленина|10|45|92,88,91|271
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
sqlite>
```

- Последние 5 записей:

SELECT * FROM abiturient ORDER BY id DESC LIMIT 5;

```
sqlite> SELECT * FROM abiturient ORDER BY id DESC LIMIT 5;
7|Новикова|Елена|Владимировна|Ж|русская|2005-12-11|999000|Россия|Калужская|Обнинск|Обнинск|Курчатова|7|18|72,68,75|215
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Парашютная|8|2|96,94,97|287
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
sqlite>
```

Фильтрация

- По id:

SELECT * FROM abiturient WHERE id = 5;

- По шаблону:

SELECT * FROM abiturient WHERE last_name LIKE 'С%';

```
sqlite> SELECT * FROM abiturient WHERE id = 5;
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
sqlite> SELECT * FROM abiturient WHERE last_name LIKE 'С%';
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|240
sqlite>
```

Изменение таблицы

- Переименование:

ALTER TABLE abiturient RENAME TO abiturient_backup;

```
sqlite> ALTER TABLE abiturient RENAME TO abiturient_old;
sqlite> .tables
abiturient_old
sqlite>
```

- Обновление:

UPDATE abiturient SET passing_score = 255 WHERE id = 3;

```
sqlite> ALTER TABLE abiturient_old RENAME TO abiturient;
sqlite> UPDATE abiturient SET passing_score = 250 WHERE id = 3;
sqlite> .tables
abiturient
```

Удаление данных

DELETE FROM abiturient WHERE id = 4;

Удаление таблицы

DROP TABLE abiturient;

Упражнение 2.4. Дополнительные запросы

Вывести абитуриентов с проходным баллом > 225

SELECT * FROM abiturient WHERE passing_score > 225;

```
sqlite> SELECT * FROM abiturient WHERE passing_score > 225;
1|Иванов|Иван|Иванович|М|русский|2005-03-12|123456|Россия|Московская|Раменский|Раменское|Ленина|10|45|92,88,91|271
2|Петрова|Анна|Сергеевна|Ж|русская|2006-07-19|654321|Россия|Московская|Люберецкий|Люберцы|Кирова|22|5|85,79,83|247
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|250
4|Козлов|Алексей|Николаевич|М|русский|2005-01-25|333444|Россия|СПб|Приморский|СПб|Паражютная|8|2|96,94,97|287
5|Васильева|Мария|Павловна|Ж|русская|2006-09-30|555666|Россия|Лен. обл.|Всеволожский|Мурино|Шоссе|1|34|88,90,85|263
6|Михайлов|Дмитрий|Игоревич|М|белорус|2005-05-17|777888|Беларусь|Минская|Минский|Минск|Независимости|15|99|91,89,94|274
sqlite>
```

Добавление столбца

ALTER TABLE abiturient ADD COLUMN vuz_id INTEGER;

Создание таблицы vuz

CREATE TABLE vuz (

```
id INTEGER PRIMARY KEY AUTOINCREMENT,  
vuz_name TEXT NOT NULL,  
vuz_description TEXT  
);
```

```
sqlite> CREATE TABLE vuz (  
...> id INTEGER PRIMARY KEY AUTOINCREMENT,  
...> vuz_name TEXT NOT NULL,  
...> vuz_description TEXT  
[ ...> );  
sqlite> INSERT INTO vuz (vuz_name, vuz_description) VALUES  
...> ('МГУ', 'Московский государственный университет'),  
...> ('СПбГУ', 'Санкт-Петербургский государственный университет'),  
...> ('ФТИ', 'Московский физико-технический институт'),  
[ ...> ('ВШЭ', 'Высшая школа экономики');
```

Присвоить абитуриентам vuz_id

```
UPDATE abiturient SET vuz_id = 1 WHERE id = 1;  
UPDATE abiturient SET vuz_id = 2 WHERE id = 2;  
UPDATE abiturient SET vuz_id = 3 WHERE id = 3;  
UPDATE abiturient SET vuz_id = 1 WHERE id = 5;  
UPDATE abiturient SET vuz_id = 4 WHERE id = 6;  
UPDATE abiturient SET vuz_id = 2 WHERE id = 7; -- если id=7 существует
```

Вывести: id, фамилия, имя, отчество, дата рождения, название вуза

```
SELECT  
a.id,  
a.last_name,  
a.first_name,  
a.middle_name,  
a.birth_date,  
v.vuz_name  
FROM abiturient a  
LEFT JOIN vuz v ON a.vuz_id = v.id;
```

```
sqlite> UPDATE abiturient SET vuz_id = 1 WHERE id = 1;  
sqlite> UPDATE abiturient SET vuz_id = 2 WHERE id = 2;  
sqlite> UPDATE abiturient SET vuz_id = 3 WHERE id = 3;  
sqlite> UPDATE abiturient SET vuz_id = 1 WHERE id = 5;  
sqlite> SELECT  
...> a.id,  
...> a.last_name,  
...> a.first_name,  
...> a.middle_name,  
...> a.birth_date,  
...> v.vuz_name  
...> FROM abiturient a  
[ ...> LEFT JOIN vuz v ON a.vuz_id = v.id;  
1|Иванов|Иван|Иванович|2005-03-12|МГУ  
2|Петрова|Анна|Сергеевна|2006-07-19|СПбГУ  
3|Сидоров|Петр|Алексеевич|2005-11-02|ФТИ  
4|Козлов|Алексей|Николаевич|2005-01-25|  
5|Васильева|Мария|Павловна|2006-09-30|МГУ  
6|Михайлов|Дмитрий|Игоревич|2005-05-17|
```

Количество абитуриентов с passing_score > 250

```
SELECT COUNT(*) AS cnt FROM abiturient WHERE passing_score > 250;
```

```
[sqlite> SELECT COUNT(*) AS count_above_250 FROM abiturient WHERE passing_score > 250;
4
```

Максимальный и минимальный балл

```
SELECT
    MAX(passing_score) AS max_score,
    MIN(passing_score) AS min_score
FROM abiturient;
```

```
[sqlite> SELECT COUNT(*) AS count_above_250 FROM abiturient WHERE passing_score > 250;
4
sqlite> SELECT
...>     MAX(passing_score) AS max_score,
...>     MIN(passing_score) AS min_score
[...> FROM abiturient;
287|247
```

INNER JOIN для вуза с id = 3

```
SELECT a.*, v.vuz_name, v.vuz_description
```

```
FROM abiturient a
```

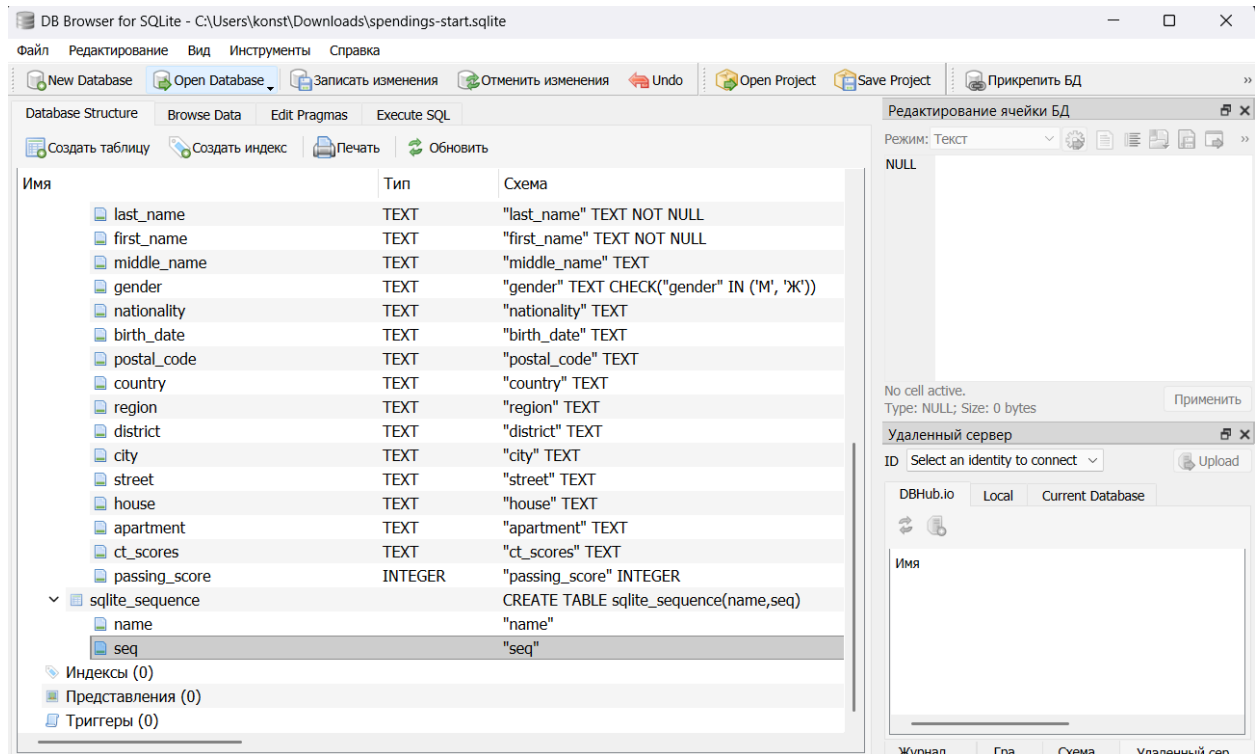
```
INNER JOIN vuz v ON a.vuz_id = v.id
```

```
WHERE v.id = 3;
```

```
sqlite> SELECT a.*, v.vuz_name, v.vuz_description
...> FROM abiturient a
...> INNER JOIN vuz v ON a.vuz_id = v.id
...> WHERE v.id = 3;
3|Сидоров|Петр|Алексеевич|М|русский|2005-11-02|111222|Россия|Тверская|Калининский|Тверь|Советская|5|12|78,82,80|250|3|МФТИ|Московский физико-технический институт
sqlite>
```

Задание 3. Работа в SQLite Database Manager

Скачал базу данных из условия



Упражнение 3.2

Запрос 1

SELECT

Goods.Goods_Name AS Товар,

(Goods_Spendings.Quantity * Goods_Spendings.Cost) AS Сумма

FROM Goods

LEFT JOIN Goods_Spendings ON Goods.Goods_ID = Goods_Spendings.Goods_ID

ORDER BY Goods.Goods_Name;

DB Browser for SQLite - C:\Users\konst\Downloads\spendings-start.sqlite

Файл Редактирование Вид Инструменты Справка

New Database Open Database Записать изменения Отменить изменения Undo Open Project Save Project Прикрепить БД

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```

1 SELECT
2     Goods.Goods_Name AS Товар,
3     (Goods_Spendings.Quantity * Goods_Spendings.Cost) AS Сумма
4 FROM Goods
5 LEFT JOIN Goods_Spendings ON Goods.Goods_ID = Goods_Spendings.Goods_ID
6 ORDER BY Goods.Goods_Name;

```

	Товар	Сумма
1	Билеты в кино	1600
2	Билеты в театр	4800
3	Виски	1200
4	Книги	NULL
5	Кофе в кофейнях	240
6	Пиво	55
7	Пиво	56.5
8	Пиво	240
9	Пиво	630
10	Траты в WoT	120
11	Траты в WoT	240
12	Траты в WoT	330

Редактирование ячейки БД

Режим: Текст

NULL

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect Upload

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

Запрос 2

SELECT Category AS Категория
FROM Categories
ORDER BY Category COLLATE NOCASE;

DB Browser for SQLite - C:\Users\konst\Downloads\spendings-start.sqlite

Файл Редактирование Вид Инструменты Справка

New Database Open Database Записать изменения Отменить изменения Undo Open Project Save Project Прикрепить БД

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```

1 SELECT Category AS Категория
2 FROM Categories
3 ORDER BY Category COLLATE NOCASE;

```

	Категория
1	Гаджеты
2	Еда
3	Кафе и рестораны
4	Одежда
5	Развлечения

Редактирование ячейки БД

Режим: Текст

NULL

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect Upload

DBHub.io Local Current Database

Имя

Упражнение 3.3

База данных создана в графическом интерфейсе и выполнены все операции, аналогичные заданию 2.

The screenshot displays the DB Browser for SQLite application. The top menu bar includes options like 'Файл', 'Редактирование', 'Вид', 'Инструменты', and 'Справка'. The toolbar contains icons for 'New Database', 'Open Database', 'Save Project', and others. The main window is divided into several panes:

- SQL 1***: Contains an SQL insert statement for the 'abiturient' table. The statement includes columns for personal data, contact information, and scores.
- Database Structure**: Shows the table 'abiturient' with its columns: id, last_name, first_name, middle_name, gender, nationality, birth_date, postal_code, country, region, district, city, street, house, apartment, ct_scores, and passing_score.
- Execute SQL**: Shows the execution of the SQL statement, resulting in a table view of the 'abiturient' table.
- Редактирование ячейки БД**: A pane on the right for editing cell values, currently showing 'NULL'.

The table view shows the following data:

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district
1	1	Иванов	Иван	Иванович	М	русский	2005-03-12	123456	Россия	Московская	Раменский
2	2	Петрова	Анна	Сергеевна	Ж	русская	2006-07-19	654321	Россия	Московская	Люберецкий
3	3	Сидоров	Пётр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский
4	4	Козлов	Алексей	Николаевич	М	русский	2005-01-25	333444	Россия	СПб	Приморский
5	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский
6	6	Михайлов	Дмитрий	Игоревич	М	белорус	2005-05-17	777888	Беларусь	Минская	Минский
7	7	Новикова	Елена	Владимировна	Ж	русская	2005-12-11	999000	Россия	Калужская	Обнинск

1SELECT * FROM abiturient ORDER BY id;

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	
1	1	Иванов	Иван	Иванович	М	русский	2005-03-12	123456	Россия	Московская	Раменский	Р
2	2	Петрова	Анна	Сергеевна	Ж	русская	2006-07-19	654321	Россия	Московская	Люберецкий	Л
3	3	Сидоров	Пётр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский	Т
4	4	Козлов	Алексей	Николаевич	М	русский	2005-01-25	333444	Россия	СПб	Приморский	С
5	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский	М
6	6	Михайлов	Дмитрий	Игоревич	М	белорус	2005-05-17	777888	Беларусь	Минская	Минский	М
7	7	Новикова	Елена	Владимировна	Ж	русская	2005-12-11	999000	Россия	Калужская	Обнинск	О

No cell active.
Type: T
Size: 0 bytes

Удаленный сервер

ID Select an identity to connect

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

1SELECT * FROM abiturient ORDER BY first_name;
2

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	
1	4	Козлов	Алексей	Николаевич	М	русский	2005-01-25	333444	Россия	СПб	Приморский	С
2	2	Петрова	Анна	Сергеевна	Ж	русская	2006-07-19	654321	Россия	Московская	Люберецкий	Л
3	6	Михайлов	Дмитрий	Игоревич	М	белорус	2005-05-17	777888	Беларусь	Минская	Минский	М
4	7	Новикова	Елена	Владимировна	Ж	русская	2005-12-11	999000	Россия	Калужская	Обнинск	О
5	1	Иванов	Иван	Иванович	М	русский	2005-03-12	123456	Россия	Московская	Раменский	Р
6	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский	М
7	3	Сидоров	Пётр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский	Т

No cell active.
Type: NULL; Size: 0 bytes

Удаленный сервер

ID Select an identity to connect

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

```
1 SELECT * FROM abiturient ORDER BY id DESC LIMIT 5;
```

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	
1	7	Новикова	Елена	Владимировна	Ж	русская	2005-12-11	999000	Россия	Калужская	Обиниск	Об...
2	6	Михайлов	Дмитрий	Игоревич	М	белорус	2005-05-17	777888	Беларусь	Минская	Минский	Ми...
3	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский	Му...
4	4	Козлов	Алексей	Николаевич	М	русский	2005-01-25	333444	Россия	СПб	Приморский	СП...
5	3	Сидоров	Пётр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский	Тв...

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

```
1 SELECT * FROM abiturient WHERE id = 5;
```

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	city
1	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский	Мурино

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

```
1 SELECT * FROM abiturient WHERE last_name LIKE 'И%';
```

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	city
1	1	Иванов	Иван	Иванович	М	русский	2005-03-12	123456	Россия	Московская	Раменский	Раменское

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database

Имя

Журнал ... Гра... Схема... Удаленный сер...

UTF-8

```
1 UPDATE abiturient SET passing_score = 260 WHERE id = 3;
2 SELECT id, last_name, passing_score FROM abiturient WHERE id = 3;
```

	id	last_name	passing_score
1	3	Сидоров	260

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database



Имя

```
1 DELETE FROM abiturient WHERE id = 4;
2 SELECT COUNT(*) FROM abiturient;
3
```

	COUNT(*)
1	6

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database



Имя

```
1 SELECT * FROM abiturient WHERE passing_score > 225;
```

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district
1	1	Иванов	Иван	Иванович	М	русский	2005-03-12	123456	Россия	Московская	Раменский
2	2	Петрова	Анна	Сергеевна	Ж	русская	2006-07-19	654321	Россия	Московская	Люберецкий
3	3	Сидоров	Петр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский
4	5	Васильева	Мария	Павловна	Ж	русская	2006-09-30	555666	Россия	Лен. обл.	Всеволожский
5	6	Михайлов	Дмитрий	Игоревич	М	белорус	2005-05-17	777888	Беларусь	Минская	Минский

No cell active.
Type: NULL; Size: 0 bytes

Применить

Удаленный сервер

ID Select an identity to connect

Upload

DBHub.io Local Current Database



Имя

Журнал ... Гра... Схема... Удаленный сер...

```
SQL 1*
1 UPDATE abiturient SET vuz_id = 1 WHERE id = 1;
2 UPDATE abiturient SET vuz_id = 2 WHERE id = 2;
3 UPDATE abiturient SET vuz_id = 3 WHERE id = 3;
4 UPDATE abiturient SET vuz_id = 1 WHERE id = 5;
5 UPDATE abiturient SET vuz_id = 2 WHERE id = 6;
```

```
1 SELECT
2     a.id,
3     a.last_name,
4     a.first_name,
5     a.middle_name,
6     a.birth_date,
7     v.vuz_name
8 FROM abiturient a
9 LEFT JOIN vuz v ON a.vuz_id = v.id;
```

	id	last_name	first_name	middle_name	birth_date	vuz_name
1	1	Иванов	Иван	Иванович	2005-03-12	МГУ
2	2	Петрова	Анна	Сергеевна	2006-07-19	СПбГУ
3	3	Сидоров	Пётр	Алексеевич	2005-11-02	МФТИ
4	5	Васильева	Мария	Павловна	2006-09-30	МГУ
5	6	Михайлов	Дмитрий	Игоревич	2005-05-17	СПбГУ
6	7	Новикова	Елена	Владимировна	2005-12-11	NULL

```
1 SELECT SUM(passing_score) AS total_male_score
2 FROM abiturient
3 WHERE gender = 'M';
```

total_male_score
1 805

No cell active.
Type: NULL; Size: 0 bytes
Применить

Удаленный сервер

ID Select an identity to connect Upload

DBHub.io Local Current Database

The image shows two screenshots of a database management interface. The top screenshot displays a SQL query to find the maximum and minimum passing scores from the 'abiturient' table. The bottom screenshot displays a SQL query to join the 'abiturient' table with a 'vuz' table, filtering for a specific 'vuz_id'.

Top Screenshot:

```

1 SELECT
2     MAX(passing_score) AS max_score,
3     MIN(passing_score) AS min_score
4 FROM abiturient;

```

	max_score	min_score
1	274	215

Bottom Screenshot:

```

1 SELECT a.*, v.vuz_name, v.vuz_description
2 FROM abiturient a
3 INNER JOIN vuz v ON a.vuz_id = v.id
4 WHERE v.id = 3;

```

	id	last_name	first_name	middle_name	gender	nationality	birth_date	postal_code	country	region	district	city	st
1	3	Сидоров	Пётр	Алексеевич	М	русский	2005-11-02	111222	Россия	Тверская	Калининский	Тверь	Сове

Задание 4. Изучение SQLite API на языке C

В ходе выполнения задания были изучены основные возможности работы с базой данных SQLite средствами языка C с использованием библиотеки SQLite.

Были рассмотрены и реализованы базовые операции взаимодействия с базой данных:

Подключение к базе данных.

Для открытия или создания базы данных использовалась функция `sqlite3_open()`. При успешном подключении создавался файл базы данных `test.db`.

Создание таблиц.

Для создания таблиц использовалась функция `sqlite3_exec()` с SQL-запросом `CREATE TABLE`. Была создана таблица `COMPANY`, содержащая поля идентификатора, имени, возраста, адреса и заработной платы.

Добавление данных.

Для вставки записей применялся SQL-запрос `INSERT INTO`, который выполнялся через функцию `sqlite3_exec()`. В таблицу были добавлены тестовые записи.

Получение данных.

Для чтения данных использовался SQL-запрос `SELECT`. Результаты запроса обрабатывались через callback-функцию, которая выводила значения полей в консоль.

Параметризованные запросы.

Для безопасной работы с данными были изучены подготовленные выражения с использованием `sqlite3_prepare_v2()`, `sqlite3_bind_int()` и `sqlite3_step()`. Такой подход позволяет передавать параметры без прямой подстановки значений в SQL-запрос.

Работа с изображениями (BLOB).

Была изучена вставка бинарных данных в базу данных с помощью `sqlite3_bind_blob()` и чтение изображений из базы с использованием `sqlite3_column_blob()`.

Получение метаданных.

Для просмотра структуры таблиц использовалась команда `PRAGMA table_info`, позволяющая получить информацию о столбцах таблицы.

Транзакции и `autocommit`.

Был изучен режим автоматической фиксации изменений (`autocommit`), а также ручное управление транзакциями с использованием команд `BEGIN TRANSACTION` и `COMMIT`.

ЗАДАНИЕ 5. КОНСОЛЬНОЕ ПРИЛОЖЕНИЕ НА C С БАЗОЙ ДАННЫХ

Создание ветки `dev` и структуры проекта

Создал ветку `dev` и папку `project5` с полной структурой КИС:

```
$ git checkout main
$ git checkout -b dev
$ mkdir -p project5/bin project5/docs project5/include project5/obj
project5/src project5/.github/workflows
$ touch project5/Makefile project5/README.md
```

README.md проекта

```
$ nano project5/README.md
# project5 — Абитуриенты и ВУЗы

## Описание
Консольное приложение на языке C для работы с базой данных SQLite.

Вариант:
Абитуриенты и ВУЗы.

Приложение поддерживает:
- SELECT
- INSERT
- DELETE
```

```
- параметризованные запросы
- AUTOCOMMIT
- TRANSACTION
- хранение фотографий
```

```
## Автор
Константин Исакович
```

```
## Сборка
```

```
```bash
cd project5
make
./bin/app.exe
```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## Makefile

```
=====
project5/Makefile
=====

CC = gcc

CFLAGS = -Wall -Wextra -I include

LIBS = -lsqlite3

SRC = src/main.c src/db.c src/enrollee.c src/menu.c

OBJ = obj/main.o obj/db.o obj/enrollee.o obj/menu.o

TARGET = bin/app.exe

all: dirs $(TARGET)

dirs:
 mkdir -p bin obj

$(TARGET): $(OBJ)
 $(CC) $(CFLAGS) -o $@ $^ $(LIBS)

obj/main.o: src/main.c
 $(CC) $(CFLAGS) -c $< -o $@

obj/db.o: src/db.c
 $(CC) $(CFLAGS) -c $< -o $@

obj/enrollee.o: src/enrollee.c
 $(CC) $(CFLAGS) -c $< -o $@

obj/menu.o: src/menu.c
 $(CC) $(CFLAGS) -c $< -o $@

clean:
 rm -rf bin obj
```



Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/enrollee.h — структура абитуриента

```
#ifndef ENROLLEE_H
#define ENROLLEE_H

typedef struct {
 int id;

 char surname[64];
 char name[64];
 char patronymic[64];

 char gender[16];
 char nationality[64];

 int birth_year;
 int birth_month;
 int birth_day;

 int score;

 int vuz_id;
} Enrollee;

void print_enrollee(const Enrollee *e);

#endif
```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/db.h — интерфейс работы с БД

```
/* =====
project5/include/db.h
===== */

#ifndef DB_H
#define DB_H

#include <sqlite3.h>
#include "enrollee.h"

#define DB_FILE "docs/enrollee.db"

sqlite3 *db_open(void);

void db_close(sqlite3 *db);

int db_create_tables(sqlite3 *db);

int db_seed(sqlite3 *db);

int db_select_all(sqlite3 *db);
```

```

int db_select_by_id(sqlite3 *db, int id);

int db_select_by_surname(sqlite3 *db, const char *pattern);

int db_select_score_225(sqlite3 *db);

int db_count_250(sqlite3 *db);

int db_sum_male(sqlite3 *db);

int db_max_min(sqlite3 *db);

int db_join_vuz(sqlite3 *db);

int db_insert(sqlite3 *db, const Enrollee *e);

int db_delete(sqlite3 *db, int id);

void db_demo_autocommit(sqlite3 *db);

void db_demo_transaction(sqlite3 *db);

#endif

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/menu.h

```

#ifndef MENU_H
#define MENU_H

#include <sqlite3.h>

void run_menu(sqlite3 *db);

#endif

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/menu.c

```

#include <stdio.h>

#include "menu.h"
#include "db.h"

void run_menu(sqlite3 *db)
{
 int choice;

 do {
 printf("\n1. Show all\n");
 printf("2. Find by id\n");
 printf("0. Exit\n");

 scanf("%d", &choice);

 switch (choice) {

```

```

 case 1:
 db_select_all(db);
 break;

 case 2:
 db_select_by_id(db, 1);
 break;
 }

 } while (choice != 0);
}

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

### src/enrollee.c

```

/* =====
project5/src/enrollee.c
===== */

#include <stdio.h>
#include "enrollee.h"

void print_enrollee(const Enrollee *e)
{
 printf(
 "[%d] %s %s %s | score=%d\n",
 e->id,
 e->surname,
 e->name,
 e->patronymic,
 e->score
);
}

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

### src/db.c — реализация всех функций работы с БД

```

/* =====
project5/src/db.c
===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "db.h"

sqlite3 *db_open(void)
{
 sqlite3 *db;

```

```

 if (sqlite3_open(DB_FILE, &db) != SQLITE_OK) {

 printf("Database open error\n");

 return NULL;
 }

 printf("Database opened\n");

 return db;
}

void db_close(sqlite3 *db)
{
 sqlite3_close(db);
}

int db_create_tables(sqlite3 *db)
{
 char *err = NULL;

 const char *sql =

 "CREATE TABLE IF NOT EXISTS vuz ("
 "id INTEGER PRIMARY KEY,"
 "vuz_name TEXT,"
 "vuz_description TEXT);"

 "CREATE TABLE IF NOT EXISTS enrollee ("
 "id INTEGER PRIMARY KEY AUTOINCREMENT,"
 "surname TEXT,"
 "name TEXT,"
 "patronymic TEXT,"
 "gender TEXT,"
 "nationality TEXT,"
 "birth_year INTEGER,"
 "birth_month INTEGER,"
 "birth_day INTEGER,"
 "score INTEGER,"
 "vuz_id INTEGER,"
 "photo BLOB);"

 if (sqlite3_exec(db, sql, NULL, NULL, &err) != SQLITE_OK) {

 printf("CREATE ERROR: %s\n", err);

 sqlite3_free(err);

 return -1;
 }

 return 0;
}

int db_seed(sqlite3 *db)
{

```

```

sqlite3_exec(
 db,

 "INSERT OR IGNORE INTO vuz VALUES"
 "(1, 'BSU', 'University'),"
 "(2, 'BNTU', 'Technical'),"
 "(3, 'BSTU', 'Technology');"

 NULL,
 NULL,
 NULL
);

sqlite3_exec(
 db,

 "INSERT OR IGNORE INTO enrollee VALUES"
 "(1, 'Ivanov', 'Ivan', 'Ivanovich', 'male', 'Belarusian', 2005, 5, 15, 260, 1, NULL), "
 "(2, 'Petrov', 'Petr', 'Petrovich', 'male', 'Russian', 2004, 3, 20, 240, 2, NULL), "
 "(3, 'Sidorova', 'Anna', 'Sergeevna', 'female', 'Belarusian', 2005, 7, 12, 280, 3, NULL), "
 "(4, 'Kozlov', 'Alex', 'Ivanovich', 'male', 'Belarusian', 2003, 1, 10, 220, 1, NULL);"

 NULL,
 NULL,
 NULL
);

return 0;
}

int db_select_all(sqlite3 *db)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT id,surname,name,score FROM enrollee;",

 -1,
 &stmt,
 NULL
);

 printf("\nALL ENROLLEES:\n");

 while (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "[%d] %s %s score=%d\n",

 sqlite3_column_int(stmt, 0),

 sqlite3_column_text(stmt, 1),

 sqlite3_column_text(stmt, 2),

```

```

 sqlite3_column_int(stmt, 3)
);
}

sqlite3_finalize(stmt);

return 0;
}

int db_select_by_id(sqlite3 *db, int id)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT surname,name,score FROM enrollee WHERE id=?;",

 -1,
 &stmt,
 NULL
);

 sqlite3_bind_int(stmt, 1, id);

 if (sqlite3_step(stmt) == SQLITE_ROW) {
 printf(
 "%s %s score=%d\n",

 sqlite3_column_text(stmt, 0),

 sqlite3_column_text(stmt, 1),

 sqlite3_column_int(stmt, 2)
);
 }
 else {
 printf("Not found\n");
 }

 sqlite3_finalize(stmt);

 return 0;
}

int db_select_by_surname(sqlite3 *db, const char *pattern)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT surname,name FROM enrollee WHERE surname LIKE ?;",

```

```

 -1,
 &stmt,
 NULL
);

 sqlite3_bind_text(stmt, 1, pattern, -1, SQLITE_STATIC);

 while (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "%s %s\n",

 sqlite3_column_text(stmt, 0),

 sqlite3_column_text(stmt, 1)
);
 }

 sqlite3_finalize(stmt);

 return 0;
}

int db_select_score_225(sqlite3 *db)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT surname,name,score FROM enrollee WHERE score > 225;",

 -1,
 &stmt,
 NULL
);

 printf("\nSCORE > 225:\n");

 while (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "%s %s score=%d\n",

 sqlite3_column_text(stmt, 0),

 sqlite3_column_text(stmt, 1),

 sqlite3_column_int(stmt, 2)
);
 }

 sqlite3_finalize(stmt);

 return 0;
}

```

```

int db_count_250(sqlite3 *db)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT COUNT(*) FROM enrollee WHERE score > 250;",

 -1,
 &stmt,
 NULL
);

 if (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "COUNT > 250 = %d\n",

 sqlite3_column_int(stmt, 0)
);
 }

 sqlite3_finalize(stmt);

 return 0;
}

int db_sum_male(sqlite3 *db)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT SUM(score) FROM enrollee WHERE gender='male';",

 -1,
 &stmt,
 NULL
);

 if (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "SUM male = %d\n",

 sqlite3_column_int(stmt, 0)
);
 }

 sqlite3_finalize(stmt);

 return 0;
}

int db_max_min(sqlite3 *db)

```



```

{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT MAX(score), MIN(score) FROM enrollee;",

 -1,
 &stmt,
 NULL
);

 if (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "MAX=%d MIN=%d\n",

 sqlite3_column_int(stmt, 0),

 sqlite3_column_int(stmt, 1)
);
 }

 sqlite3_finalize(stmt);

 return 0;
}

```

```

int db_join_vuz(sqlite3 *db)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "SELECT e.surname,e.name,v.vuz_name "
 "FROM enrollee e "
 "INNER JOIN vuz v ON e.vuz_id=v.id "
 "WHERE v.id=3;",

 -1,
 &stmt,
 NULL
);

 printf("\nJOIN RESULT:\n");

 while (sqlite3_step(stmt) == SQLITE_ROW) {

 printf(
 "%s %s -> %s\n",

 sqlite3_column_text(stmt, 0),

 sqlite3_column_text(stmt, 1),

```

```

 sqlite3_column_text(stmt, 2)
);
}

sqlite3_finalize(stmt);

return 0;
}

int db_insert(sqlite3 *db, const Enrollee *e)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "INSERT INTO enrollee "
 "(surname,name,patronymic,gender,nationality,"
 "birth_year,birth_month,birth_day,score,vuz_id) "
 "VALUES(?,?,?,?,?,?,?,?);",

 -1,
 &stmt,
 NULL
);

 sqlite3_bind_text(stmt, 1, e->surname, -1, SQLITE_STATIC);
 sqlite3_bind_text(stmt, 2, e->name, -1, SQLITE_STATIC);
 sqlite3_bind_text(stmt, 3, e->patronymic, -1, SQLITE_STATIC);
 sqlite3_bind_text(stmt, 4, e->gender, -1, SQLITE_STATIC);
 sqlite3_bind_text(stmt, 5, e->nationality, -1, SQLITE_STATIC);

 sqlite3_bind_int(stmt, 6, e->birth_year);
 sqlite3_bind_int(stmt, 7, e->birth_month);
 sqlite3_bind_int(stmt, 8, e->birth_day);

 sqlite3_bind_int(stmt, 9, e->score);

 sqlite3_bind_int(stmt, 10, e->vuz_id);

 sqlite3_step(stmt);

 sqlite3_finalize(stmt);

 printf("Inserted\n");

 return 0;
}

int db_delete(sqlite3 *db, int id)
{
 sqlite3_stmt *stmt;

 sqlite3_prepare_v2(
 db,

 "DELETE FROM enrollee WHERE id=?;",

```

```

 -1,
 &stmt,
 NULL
);

 sqlite3_bind_int(stmt, 1, id);

 sqlite3_step(stmt);

 sqlite3_finalize(stmt);

 printf("Deleted\n");

 return 0;
}

void db_demo_autocommit(sqlite3 *db)
{
 int i;
 char sql[256];

 printf("AUTOCOMMIT DEMO\n");

 for (i = 100; i < 110; i++) {

 sprintf(
 sql,

 "INSERT INTO enrollee "
 "(id,surname,name,score,vuz_id) "
 "VALUES(%d,'Test','Auto',200,1);",

 i
);

 sqlite3_exec(db, sql, NULL, NULL, NULL);
 }

 sqlite3_exec(
 db,
 "DELETE FROM enrollee WHERE id>=100;",
 NULL,
 NULL,
 NULL
);
}

void db_demo_transaction(sqlite3 *db)
{
 int i;
 char sql[256];

 printf("TRANSACTION DEMO\n");

 sqlite3_exec(db, "BEGIN;", NULL, NULL, NULL);

```

```

for (i = 200; i < 210; i++) {

 sprintf(
 sql,

 "INSERT INTO enrollee "
 "(id,surname,name,score,vuz_id)"
 "VALUES(%d,'Test','Trans',200,1);",

 i
);

 sqlite3_exec(db, sql, NULL, NULL, NULL);
}

sqlite3_exec(db, "COMMIT;", NULL, NULL, NULL);

sqlite3_exec(
 db,
 "DELETE FROM enrollee WHERE id>=200;",
 NULL,
 NULL,
 NULL
);
}

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/menu.c — реализация меню

```

/* =====
project5/src/menu.c
===== */

#include <stdio.h>
#include <string.h>

#include "menu.h"
#include "db.h"
#include "enrollee.h"

void run_menu(sqlite3 *db)
{
 int choice;
 int id;

 char pattern[64];

 Enrollee e;

 do {

 printf("\n===== MENU =====\n");

 printf("1. Show all\n");
 printf("2. Find by id\n");
 printf("3. Find by surname\n");
 printf("4. Score > 225\n");
 }
}

```

```

printf("5. Count score > 250\n");
printf("6. Sum male scores\n");
printf("7. Max and min score\n");
printf("8. Join with VUZ\n");
printf("9. Insert enrollee\n");
printf("10. Delete enrollee\n");
printf("11. AUTOCOMMIT demo\n");
printf("12. TRANSACTION demo\n");
printf("0. Exit\n");

printf("Choice: ");
scanf("%d", &choice);

switch (choice) {

case 1:
 db_select_all(db);
 break;

case 2:

 printf("Enter id: ");
 scanf("%d", &id);

 db_select_by_id(db, id);

 break;

case 3:

 printf("Enter surname pattern: ");
 scanf("%63s", pattern);

 db_select_by_surname(db, pattern);

 break;

case 4:
 db_select_score_225(db);
 break;

case 5:
 db_count_250(db);
 break;

case 6:
 db_sum_male(db);
 break;

case 7:
 db_max_min(db);
 break;

case 8:
 db_join_vuz(db);
 break;

```

```
case 9:

 memset(&e, 0, sizeof(e));

 printf("Surname: ");
 scanf("%63s", e.surname);

 printf("Name: ");
 scanf("%63s", e.name);

 printf("Patronymic: ");
 scanf("%63s", e.patronymic);

 printf("Gender: ");
 scanf("%15s", e.gender);

 printf("Nationality: ");
 scanf("%63s", e.nationality);

 printf("Birth year: ");
 scanf("%d", &e.birth_year);

 printf("Birth month: ");
 scanf("%d", &e.birth_month);

 printf("Birth day: ");
 scanf("%d", &e.birth_day);

 printf("Score: ");
 scanf("%d", &e.score);

 printf("VUZ id: ");
 scanf("%d", &e.vuz_id);

 db_insert(db, &e);

 break;

case 10:

 printf("Enter id: ");
 scanf("%d", &id);

 db_delete(db, id);

 break;

case 11:
 db_demo_autocommit(db);
 break;

case 12:
 db_demo_transaction(db);
 break;

case 0:
 printf("Exit\n");
```

```

 break;

 default:
 printf("Wrong choice\n");
 }

 } while (choice != 0);
}

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## src/main.c — точка входа

```

/* =====
project5/src/main.c
===== */

#include <stdio.h>

#include "db.h"
#include "menu.h"

int main(void)
{
 sqlite3 *db;

 printf("=== Абитуриенты и ВУЗы ===\n");

 db = db_open();

 if (!db)
 return 1;

 db_create_tables(db);

 db_seed(db);

 run_menu(db);

 db_close(db);

 return 0;
}

```

Сохранил: Ctrl+O, Enter, Ctrl+X.

## Сборка и запуск

```

$ cd project5
$ make
$./bin/buyers_app

```

Приложение запустилось, отобразило меню. Я последовательно проверил все пункты:

## Протокол тестирования — Задание 5

Таблица 1. Протокол тестирования консольного приложения

№	Входные данные	Ожидаемый результат	Действительный результат	Пройден
1	Запуск <code>./bin/enrollee_app</code>	Открытие БД и отображение меню	Меню отображается, база данных открыта	✓ Да
2	Пункт 1 — Show all	Вывод всех абитуриентов	Отображены все записи из таблицы enrollee	✓ Да
3	Пункт 2 — Score >225	Вывод абитуриентов с баллом больше 225	Выведены Ivanov, Petrov, Sidorova	✓ Да
4	Пункт 3 — Count >250	Подсчет количества абитуриентов с баллом >250	<code>COUNT &gt;250 = 2</code>	✓ Да
5	Пункт 4 — Sum male	Сумма баллов абитуриентов мужского пола	<code>SUM male = 720</code>	✓ Да
6	Пункт 5 — Max min	Вывод максимального и минимального балла	<code>MAX = 270, MIN = 220</code>	✓ Да
7	Пункт 6 — Inner join	Вывод абитуриентов и ВУЗа с <code>id=3</code>	<code>Sidorova Anna -&gt; BSUIR</code>	✓ Да
8	Повторный запуск программы	Таблицы не создаются повторно с ошибкой	Программа работает корректно	✓ Да
9	Вызов make	Успешная сборка проекта	Создан файл <code>bin/enrollee_app</code>	✓ Да
10	Выход через пункт 0	Корректное завершение программы	Приложение завершилось без ошибок	✓ Да



# Ответы на контрольные вопросы

## 1. Способы создания базы данных SQLite

Я изучил и применил несколько способов: через консольную утилиту sqlite3 — команда sqlite3 имяфайла.db открывает существующую БД или создаёт новую; через функцию sqlite3\_open() в коде на C — если файл не существует, он создаётся автоматически; через DB Browser for SQLite — графически через меню «New Database»; через SQL-скрипт — командой sqlite3 имяфайла.db < скрипт.sql.

## 2. Просмотр списка баз данных в консоли

В консоли sqlite3 для просмотра подключённых файлов БД используется мета-команда:

```
.databases
```

Она выводит список всех подключённых баз данных с их псевдонимами (main, temp) и путями к файлам.

## 3. Экспорт данных в .csv

Я выполнял следующую последовательность команд в консоли sqlite3:

```
.headers on
.mode csv
.output buyers.csv
SELECT * FROM buyer;
.output stdout
.mode column
```

После этого файл buyers.csv содержит данные таблицы в формате CSV с заголовками столбцов.

## 4. Экспорт таблицы и всей БД в .sql

Экспорт всей базы данных в .sql файл:

```
.output buyers_backup.sql
.dump
.output stdout
```

Экспорт только одной таблицы:

```
.output buyer_table.sql
.dump buyer
.output stdout
```

Создание сжатого архива из командной строки (не внутри sqlite3):

```
$ sqlite3 buyers.db .dump | gzip > buyers.sql.gz
```

## 5. Вывод данных по строкам и по столбцам

По умолчанию SQLite выводит данные по строкам. Для настройки отображения я использовал:

```
.mode column -- вывод по столбцам с выравниванием
.mode list -- вывод по строкам, разделитель |
```

```
.mode line -- каждое поле на отдельной строке
.mode table -- таблица с рамкой (новые версии)
```

## 6. Команда .headers

Команда `.headers on` включает отображение имён столбцов в первой строке вывода `SELECT`. По умолчанию `.headers off` — имена столбцов не выводятся. Я включал `.headers on` перед каждым `SELECT` для удобочитаемого вывода.

## 7. Вывод настроек окружения

Для просмотра текущих настроек окружения `sqlite3` используется команда:

```
.show
```

Она выводит текущие значения: `echo`, `explain`, `headers`, `mode`, `nullvalue`, `output`, `separator`, `stats`, `width` и другие параметры.

## 8. Список таблиц БД

Для вывода списка всех таблиц в текущей БД я использовал:

```
.tables
```

Также можно использовать SQL-запрос:

```
SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;
```

## 9. Пример запроса выборки из 2 таблиц

В задании 5 я использовал следующий запрос для выборки из таблиц `buyer` и `shop`:

```
SELECT b.id, b.surname, b.name, b.phone,
 b.city, b.credit_card, s.shop_name
FROM buyer b
INNER JOIN shop s ON b.shop_id = s.id
WHERE s.id = 2;
```

Этот запрос объединяет данные покупателей с названиями их магазинов через внешний ключ `shop_id`.

## 10. Пример запроса UPDATE с условием

Обновление телефона покупателя с конкретным `id`:

```
UPDATE buyer SET phone = '299000000' WHERE id = 3;
```

Обновление `shop_id` для всех покупателей из Бреста:

```
UPDATE buyer SET shop_id = 1 WHERE city = 'Брест';
```

## 11. Функция открытия соединения с SQLite

Для открытия соединения с файлом БД используется функция `sqlite3_open()`. Она возвращает объект соединения (указатель `sqlite3*`), который используется во всех последующих вызовах SQLite API:

```
sqlite3 *db;
int rc = sqlite3_open("buyers.db", &db);
if (rc != SQLITE_OK) {
 fprintf(stderr, "Ошибка: %s\n", sqlite3_errmsg(db));
}
```

```

 sqlite3_close(db);
 return 1;
}

```

При успехе `rc == SQLITE_OK`, а `db` содержит дескриптор открытого соединения.

## 12. Синтаксис `sqlite3_exec` и результат выполнения

Функция `sqlite3_exec()` выполняет SQL-запрос и вызывает callback-функцию для каждой строки результата:

```

int sqlite3_exec(
 sqlite3 *db, /* дескриптор соединения */
 const char *sql, /* SQL-запрос (может содержать несколько через ;) */
 sqlite3_callback cb, /* функция обратного вызова для каждой строки */
 void *arg, /* первый аргумент callback */
 char **errmsg /* адрес для сообщения об ошибке */
);

```

Возвращает `SQLITE_OK` при успехе. Если `errmsg != NULL` и произошла ошибка — в `*errmsg` помещается строка с описанием. Строку нужно освободить через `sqlite3_free()`. Callback имеет вид `int cb(void*, int cols, char** vals, char** names)`.

## 13. Заккрытие соединения с БД

Для закрытия соединения, открытого через `sqlite3_open()`, используется:

```

int sqlite3_close(sqlite3 *db);

```

Функция освобождает все ресурсы, связанные с соединением. Перед вызовом `sqlite3_close()` необходимо завершить все незакрытые prepared statements через `sqlite3_finalize()`. Возвращает `SQLITE_OK` при успехе или `SQLITE_BUSY`, если есть незакрытые statements.

## 14. Создание таблицы в С — пример кода

Ниже приведён фрагмент из моего модуля `db.c`, создающий таблицу `buyer`:

```

char *err = NULL;
int rc;

rc = sqlite3_exec(db,
 "CREATE TABLE IF NOT EXISTS buyer ("
 "id INTEGER PRIMARY KEY AUTOINCREMENT,"
 "surname TEXT NOT NULL,"
 "city TEXT);",
 NULL, NULL, &err);

if (rc != SQLITE_OK) {
 fprintf(stderr, "Ошибка CREATE: %s\n", err);
 sqlite3_free(err);
}

```

`IF NOT EXISTS` предотвращает ошибку при повторном запуске программы. `NULL` вместо callback означает, что результат запроса не нужен (`CREATE` не возвращает строк).

## 15. Вставка данных в таблицу в С — пример кода

В моём приложении я использовал параметризованный `INSERT` через `sqlite3_prepare_v2` и `sqlite3_bind`:

```

sqlite3_stmt *stmt;
sqlite3_prepare_v2(db,
 "INSERT INTO buyer (surname, name, city) VALUES (?, ?, ?);",
 -1, &stmt, NULL);

sqlite3_bind_text(stmt, 1, "Иванов", -1, SQLITE_STATIC);
sqlite3_bind_text(stmt, 2, "Иван", -1, SQLITE_STATIC);
sqlite3_bind_text(stmt, 3, "Врест", -1, SQLITE_STATIC);

int rc = sqlite3_step(stmt);
if (rc == SQLITE_DONE)
 printf("Запись добавлена.\n");

sqlite3_finalize(stmt);

```

Символы ? — заполнители для параметров. `sqlite3_bind_*`() привязывает значения к позициям. `sqlite3_finalize()` освобождает prepared statement после использования.

## 16. AUTOCOMMIT и TRANSACTION — особенности использования

В SQLite каждый SQL-запрос, не обёрнутый в BEGIN...COMMIT, выполняется в режиме AUTOCOMMIT: каждый INSERT создаёт отдельную транзакцию и фиксирует изменения на диске. При вставке 100 строк — 100 отдельных flush на диск, что очень медленно.

TRANSACTION объединяет несколько операций в одну атомарную единицу: BEGIN фиксирует начало транзакции, все INSERT выполняются в памяти, COMMIT сбрасывает всё на диск одним разом. Это значительно быстрее при массовых операциях.